



第8章 输入输出技术

14:09

第8章 输入输出技术



8.1 I/O接口概述

8.2 简单I/O接口芯片（重点）

8.3 基本输入/输出方法（重难点）

14:09

第8章 输入输出技术



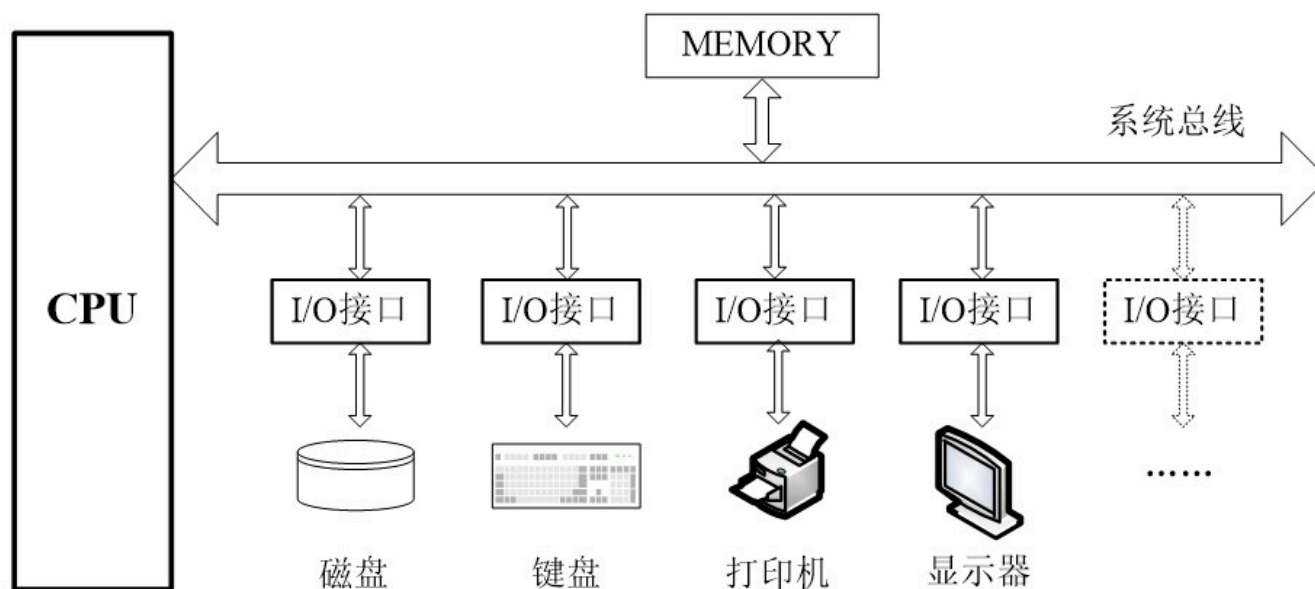
8.1 I/O接口概述

8.2 简单I/O接口芯片（重点）

8.3 基本输入/输出方法（重难点）

14:09

8.1 I/O接口概述



I/O接口：将**CPU**和外围设备连接起来实现信息交换的缓冲电路称为**I/O接口（Interface）**电路，简称“**I/O接口**”。

8.1 I/O接口概述



I/O接口分类

- 按数据传送方式：并行接口、串行接口
- 按功能选择的灵活性：可编程接口、不可编程接口
- 按应用的通用性：通用接口、专用接口
- 按数据控制的方式：程序型接口、**DMA**型接口

8.1 I/O接口概述



- 一、I/O接口的基本功能
- 二、I/O接口的基本结构
- 三、I/O端口的编址方式
- 四、I/O端口的地址译码

8.1 I/O接口概述



一、I/O接口的基本功能

二、I/O接口的基本结构

三、I/O端口的编址方式

四、I/O端口的地址译码

一、I/O接口的基本功能



- 缓冲、隔离和锁存功能
- 信息格式与电平转换功能
- 信息交换的应答联络功能
- 译码寻址外设功能

基本原则：输入要缓冲、输出要锁存、
输入/输出要隔离

8.1 I/O接口概述



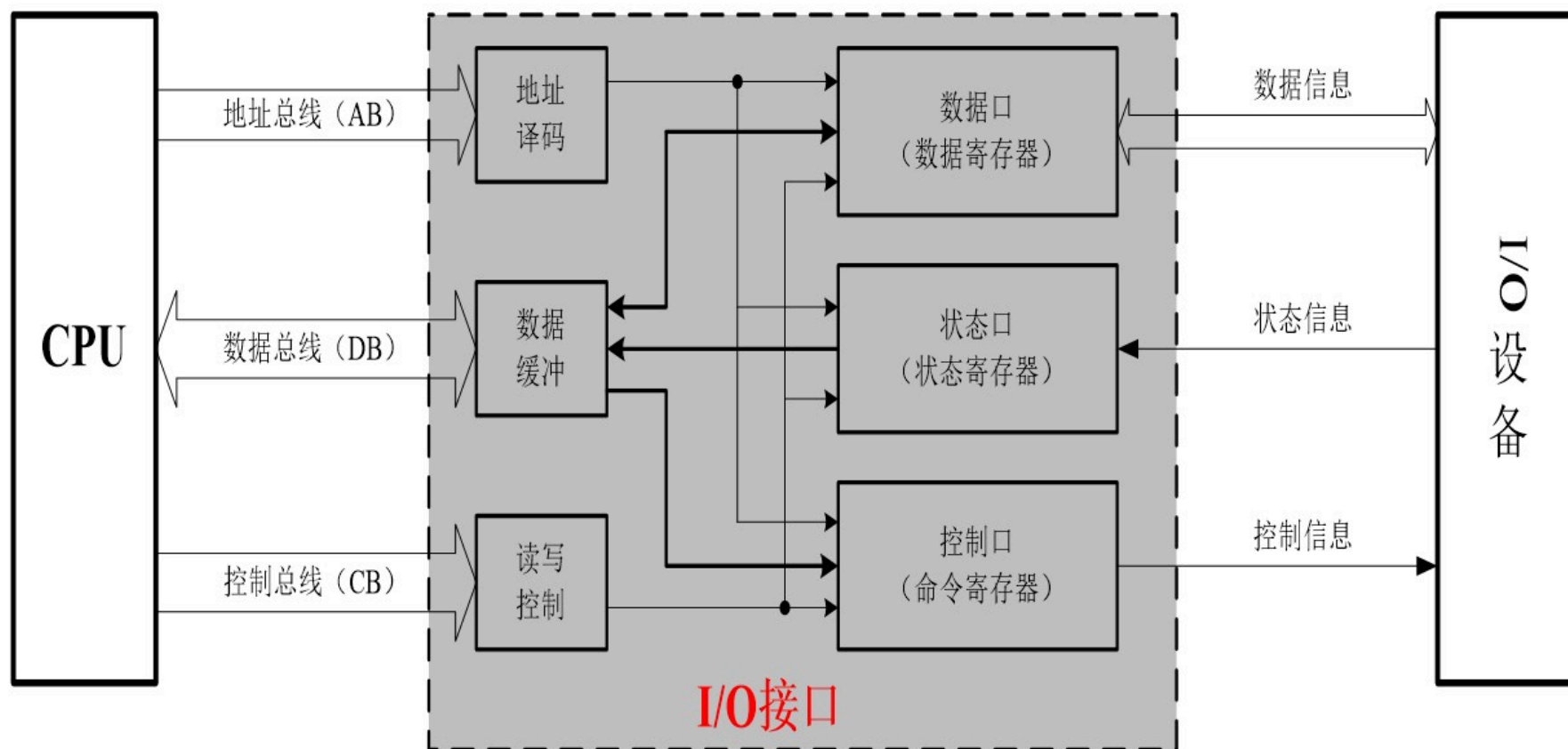
一、I/O接口的基本功能

二、I/O接口的基本结构

三、I/O端口的编址方式

四、I/O端口的地址译码

二、I/O接口的基本结构



二、I/O接口的基本结构



- 数据（端）口=数据寄存器：数据信息
- 状态（端）口=状态寄存器：状态信息
- 控制（端）口=命令寄存器：控制信息

8.1 I/O接口概述



一、I/O接口的基本功能

二、I/O接口的基本结构

三、I/O端口的编址方式

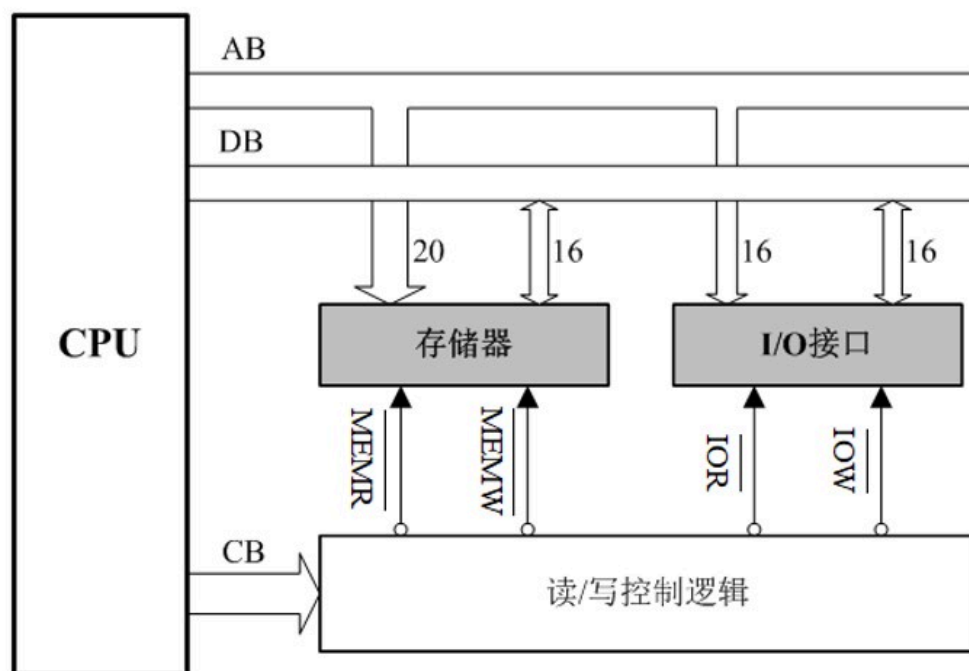
四、I/O端口的地址译码

三、I/O端口的编址方式



1、独立编址

相对存储器而言，将存储器地址空间和I/O接口寄存器地址空间分开设置，互不影响。**Intel**系列。



三、I/O端口的编址方式



独立编址方式的特点：

- 存储器和**I/O**接口各自拥有独立的地址空间。
- 存储器和**I/O**接口的读写控制逻辑相互独立。
- **CPU**的访内指令和访外指令不同。
- **I/O**端口的地址码较短(**8位或16位**)，译码电路比较简单，从而使**IN/OUT**指令的执行速度较快。

缺点：

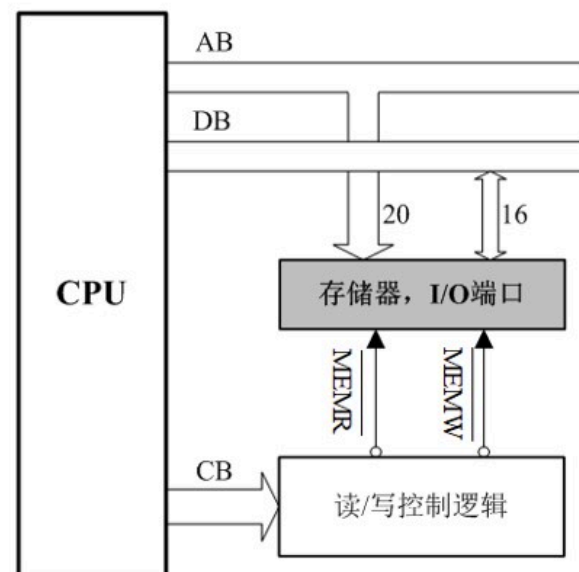
需要专用的访外指令，程序设计的灵活性较差。

三、I/O端口的编址方式



2、统一编址

又称存储器映像编址，将所有I/O接口电路中的寄存器作为存储单元对待，并给每一个寄存器分配相应的存储器地址。这样，**CPU**访问外设就象访问存储器一样，所不同的仅是地址而已。**MCS—51**系列，**680X0**系列，**ARM**系列等。



三、I/O端口的编址方式



统一编址方式的特点：

- 访问存储器的指令也可用于外设的输入/输出操作。
- I/O接口和存储器共用译码电路和读写控制逻辑。
- CPU无需产生访内操作和访外操作的控制信号。
- I/O端口的地址空间可大可小。

缺点：

- 不能充分利用地址空间，地址空间的分配可能潜藏隐患，影响存储器空间的容量；
- CPU与I/O端口交换数据的指令执行时间较长。

8.1 I/O接口概述



一、I/O接口的基本功能

二、I/O接口的基本结构

三、I/O端口的编址方式

四、I/O端口的地址译码

四、I/O端口的地址译码



与存储器单元的地址译码方法相同。

统一编址：完全相同。

独立编址：方法相同，通常I/O端口地址比存储器单元地址短，因此参与译码的地址线较少。

➤ **8位端口（静态地址）：**地址线**A7~A0**（**8根线**，其余忽略），**00H~FFH**，可直接在**IN/OUT**指令中出现。

➤ **16位端口（动态地址）：**地址线**A15~A0**（**16根线**，其余忽略），**0000H~FFFFH**，端口地址必须先放在**DX**寄存器中，才能使用**IN/OUT**指令。

四、I/O端口的地址译码



IN AL, 20H ; 20H是8位端口地址,
可直接在指令中作为操作数使用

MOV DX, 13F8H

OUT DX, AL ; 13F8H是16位端口地
址, 应先将其装入**DX**, 才能被寻址

第8章 输入输出技术



8.1 I/O接口概述

8.2 简单I/O接口芯片（重点）

8.3 基本输入/输出方法（重难点）

14:09

8.2 简单I/O接口芯片（重点）



一、74LS373锁存器

二、74LS244缓冲器

三、74LS245收发器

8.2 简单I/O接口芯片（重点）

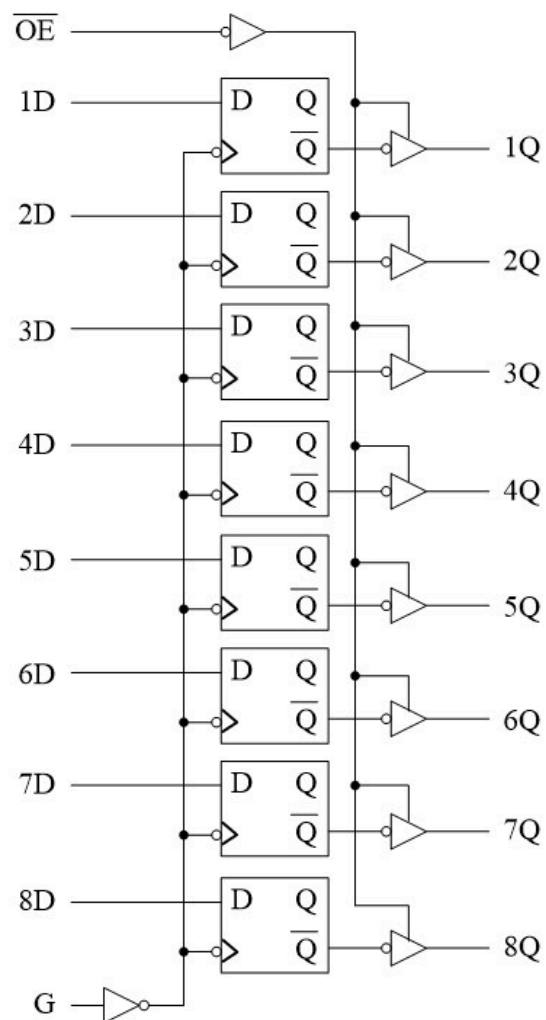


一、74LS373锁存器

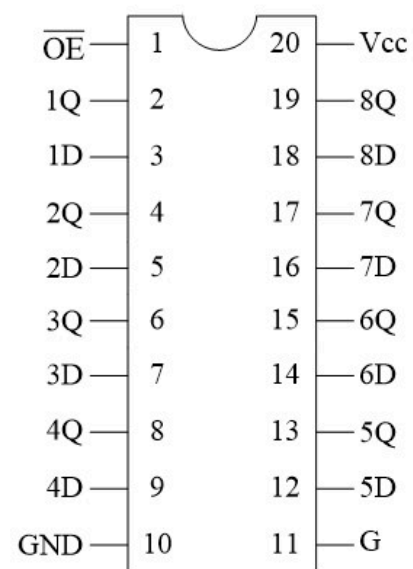
二、74LS244缓冲器

三、74LS245收发器

一、74LS373锁存器



(a) 逻辑电路



(b) 引脚图

一、74LS373锁存器



74LS373真值表

输入门控G	$\overline{OE}$ 输出允许	输入D	输出Q
H	L	L	L
H	L	H	H
L	L	D ₀ (保持不变)	Q ₀ (原状态)
L	H	D ₀ (保持不变)	Z (高阻态)
H	H	X	Z (高阻态)

74LS373可被用于锁存地址信息、数据信息和状态信息等，更多用于锁存地址信息。

8.2 简单I/O接口芯片（重点）

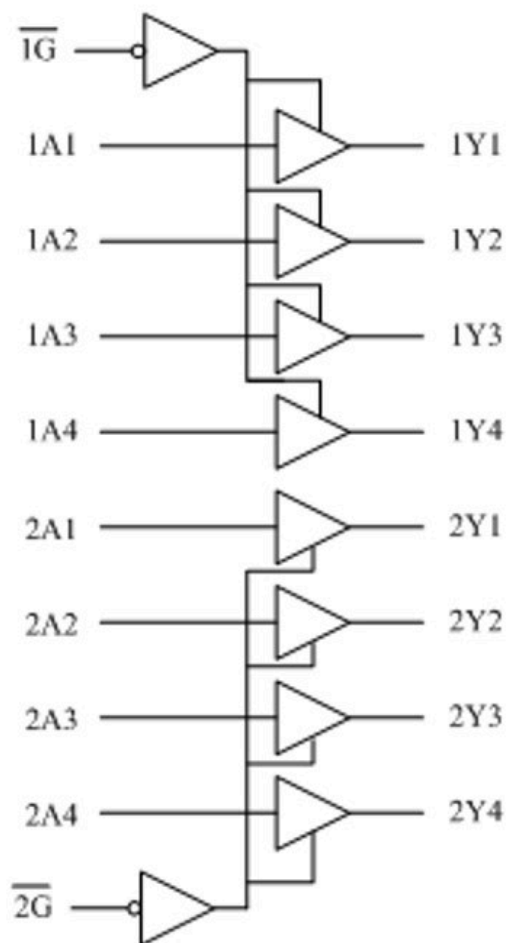


一、74LS373锁存器

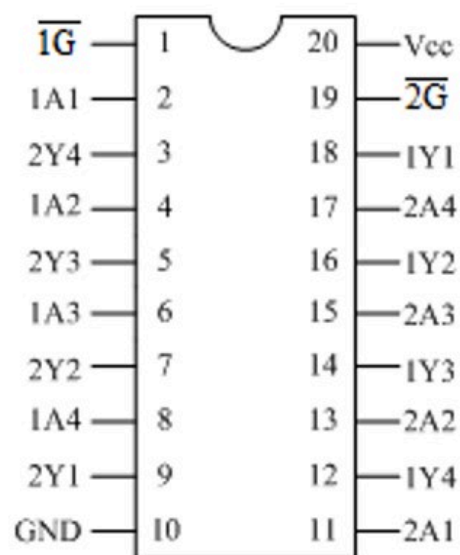
二、74LS244缓冲器

三、74LS245收发器

二、74LS244缓冲器



(a) 逻辑电路



(b) 引脚图

输入		输出
$\overline{G}$	A	Y
L	L	L
L	H	H
H	X	Z

(c) 真值表

8.2 简单I/O接口芯片（重点）



一、74LS373锁存器

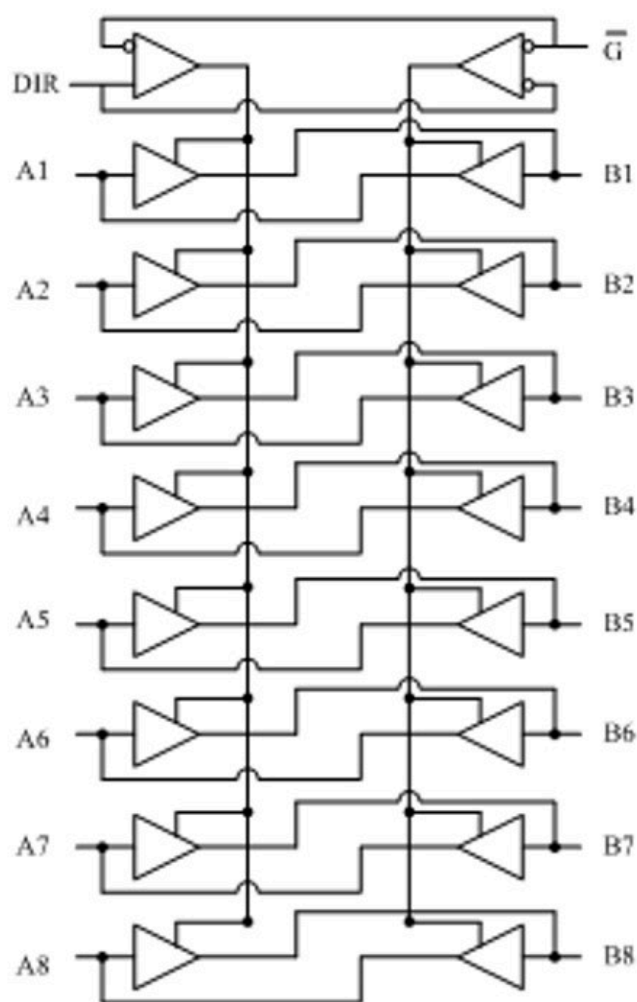
二、74LS244缓冲器

三、74LS245收发器

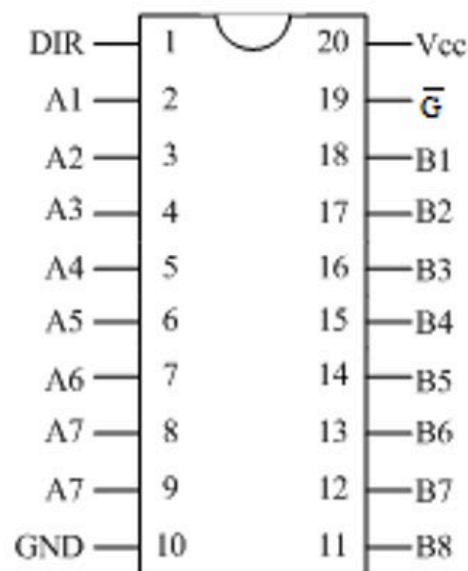
三、74LS245收发器



74LS245常用于数据的双向传送、缓冲和驱动。



(a) 逻辑电路

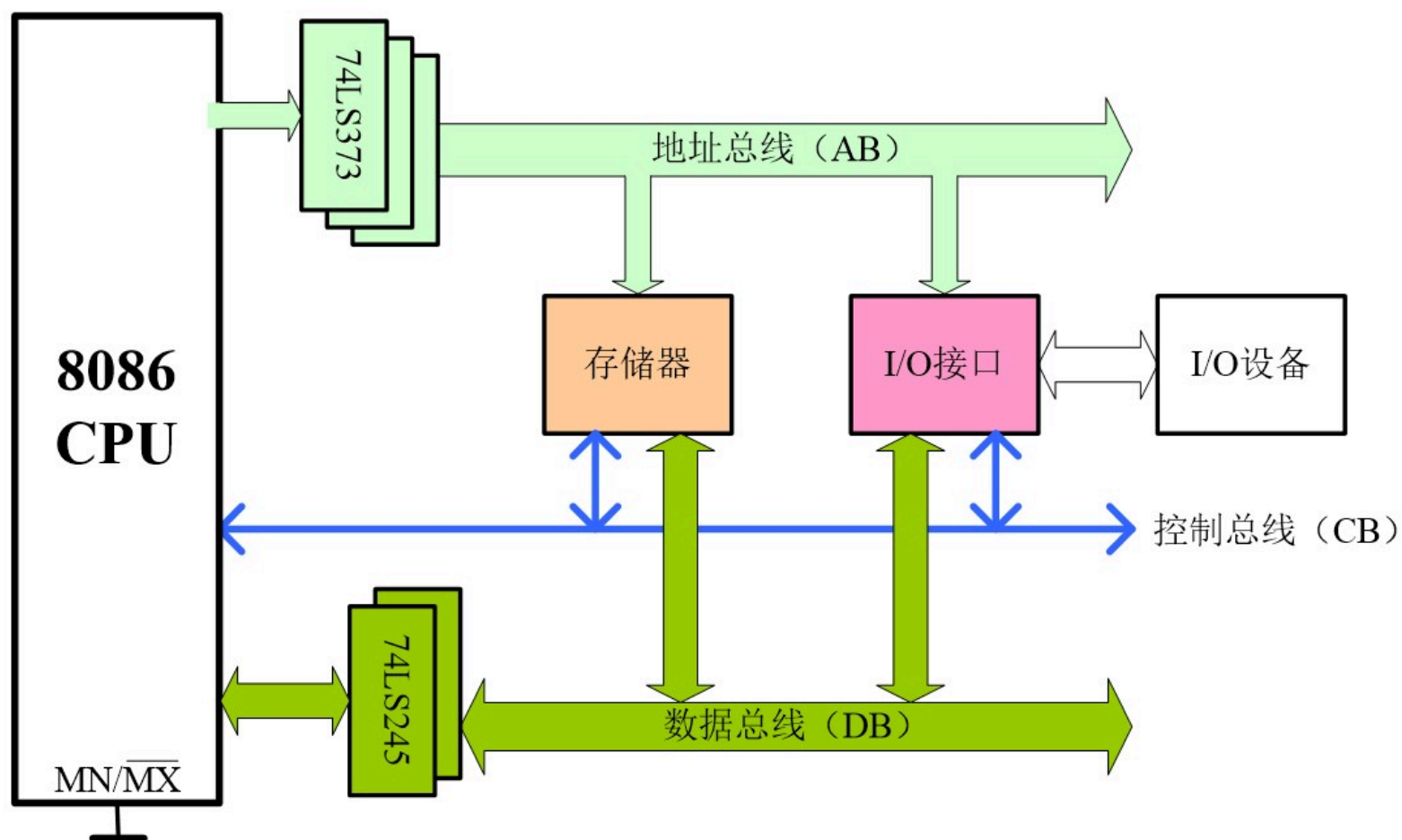


(b) 引脚图

$\overline{G}$	DIR	操作
L	L	$A \leftarrow B$
L	H	$B \leftarrow A$
H	X	Z

(c) 真值表

8.2 简单I/O接口芯片（重点）



第8章 输入输出技术



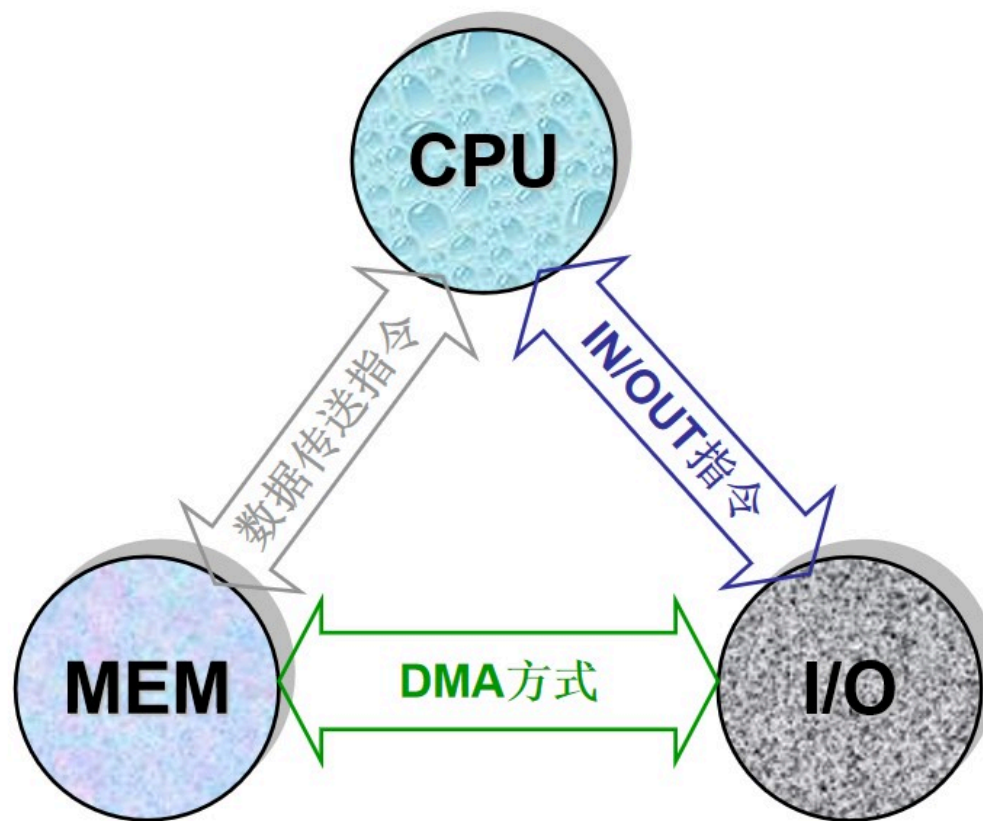
8.1 I/O接口概述

8.2 简单I/O接口芯片（重点）

8.3 基本输入/输出方法（重难点）

14:09

8.3 基本输入/输出方法（重难点）



8.3 基本输入/输出方法（重难点）



一、程序控制的输入/输出（重难点）

二、直接存储器存取方式（DMA）

8.3 基本输入/输出方法（重难点）



一、程序控制的输入/输出（重难点）

二、直接存储器存取方式（DMA）

一、程序控制的输入/输出（重难点）



- 传送过程以**CPU**为中心，通过程序(数据传送指令和**IN/OUT**指令)完成数据传送
- 传送路径必须经过**CPU**内部的寄存器
- 传送速度不高，响应比较慢

- 1、无条件传送
- 2、查询方式传送
- 3、中断方式传送

一、程序控制的输入/输出



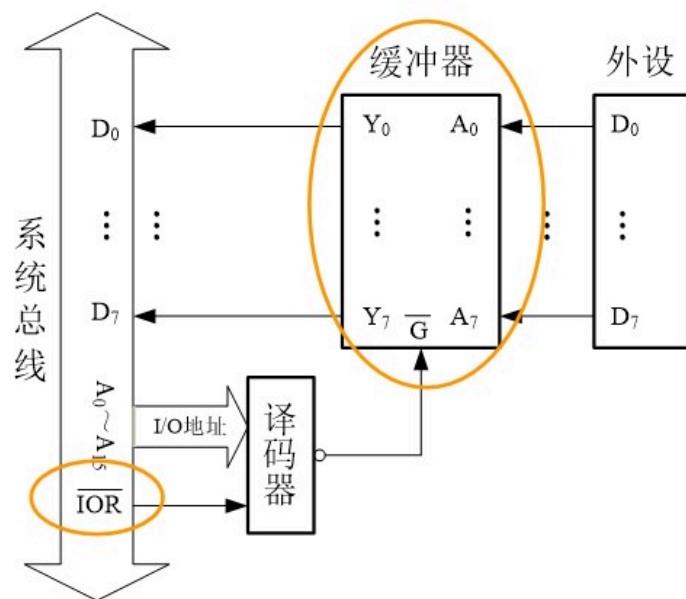
1、无条件传送

- 又称同步传送，**CPU**可以随时与**I/O**进行数据传送，不依赖于任何条件
- 对于输入口，**CPU**总是认为外设数据已经准备好，可读
- 对于输出口，**CPU**总是假设外设数据端口已空，可写
- 不需要在**CPU**和**I/O**之间建立任何握手信号，所需硬件很少，软件简单
- 适用于开关、继电器、步进电机、**LED**、数码管显示器等

一、程序控制的输入/输出（无条件传送）

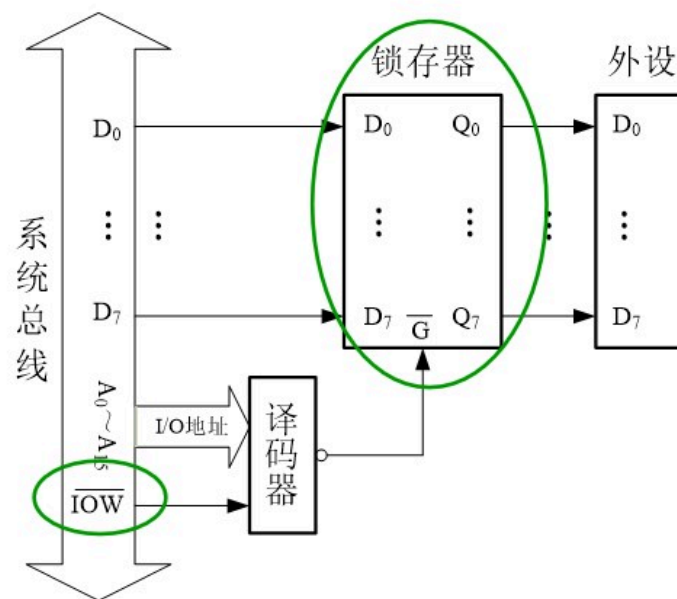


输入接口设计



```
MOV DX, IN_PORT
IN AL, DX
```

输出接口设计



```
MOV DX, OUT_PORT
OUT DX, AL
```

一、程序控制的输入/输出（无条件传送）



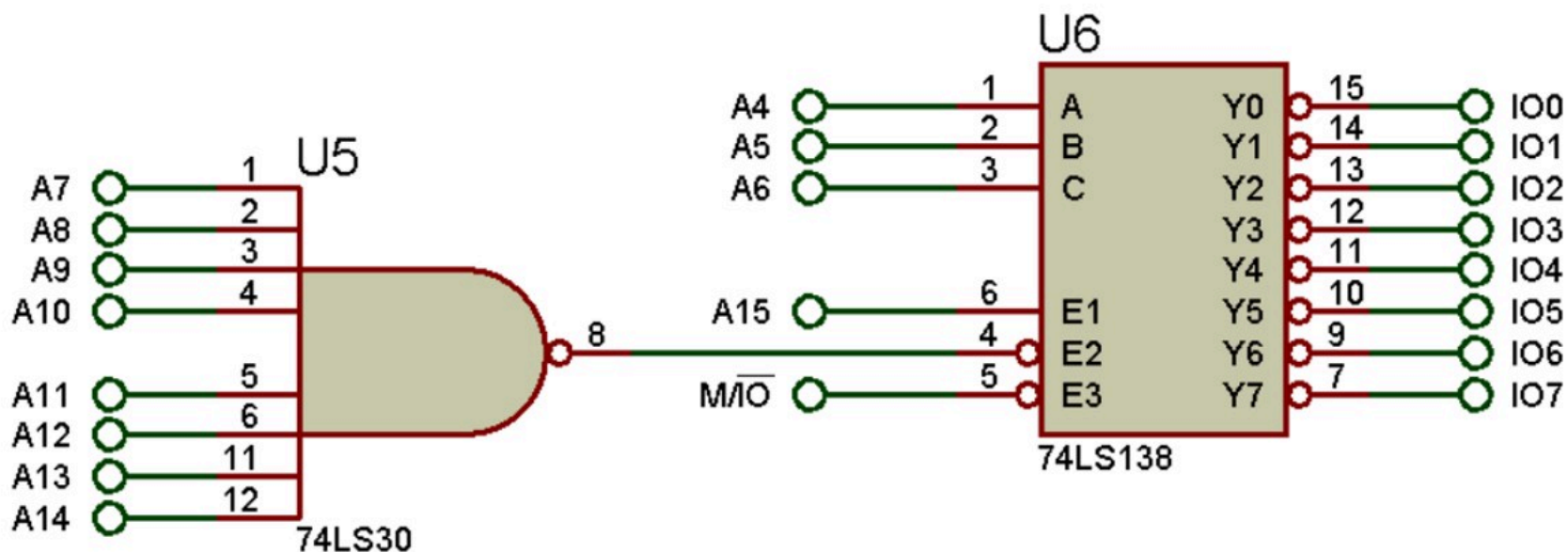
例1：设计原理图，利用**74LS373**连接**8**个**LED**灯，利用**74HC245**连接**8**个开关，编写接口程序实现每个开关控制一个**LED**灯。（群文件\仿真实例\简单I/O接口.rar）

一、程序控制的输入/输出（无条件传送）

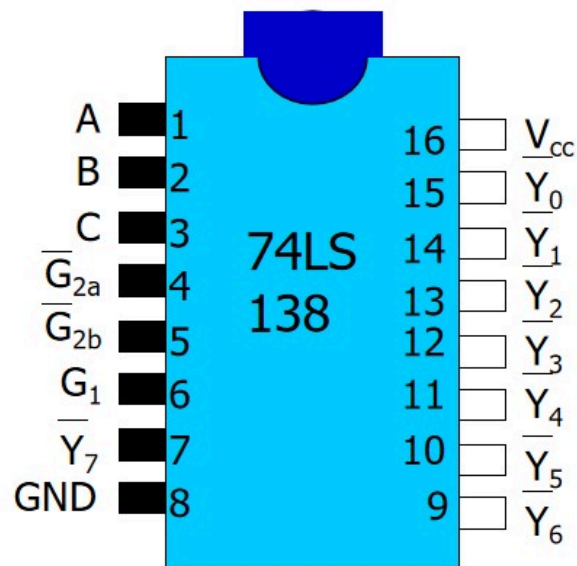


① I/O端口的地址译码电路的分析

I/O端口的地址译码电路如下图所示：

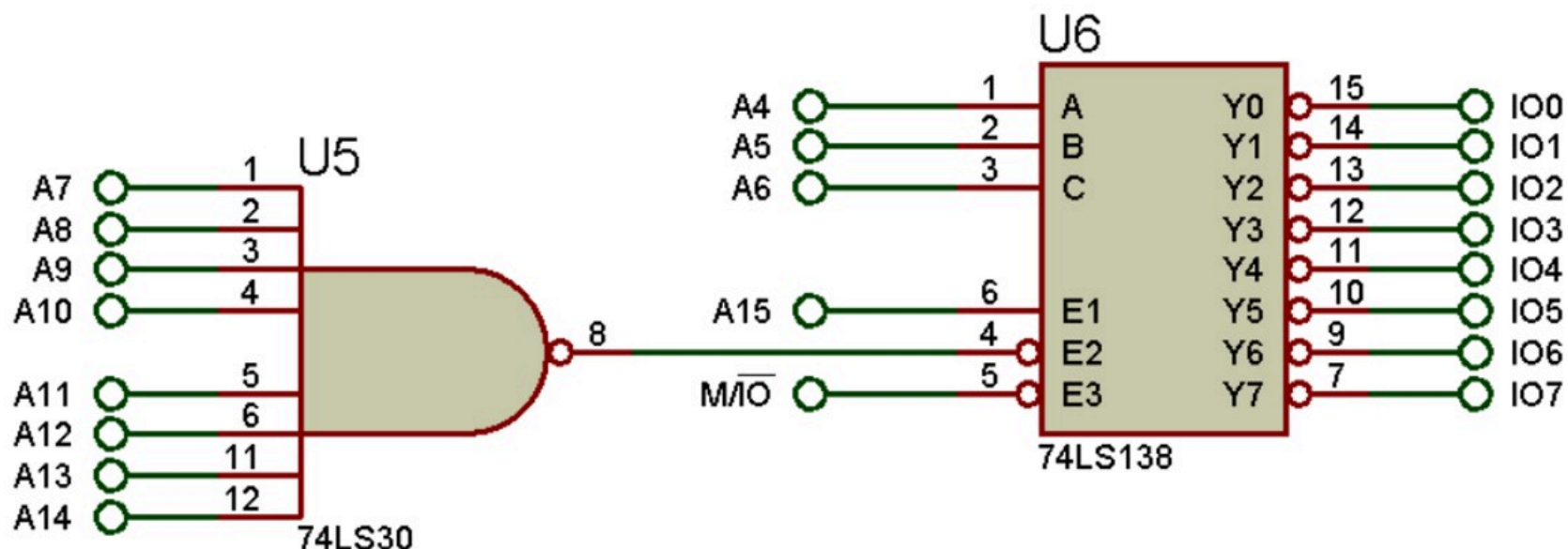


74LS138译码器(即Intel 8205译码器)



G1	$\overline{G}_{2a}$	$\overline{G}_{2b}$	C	B	A	$\overline{Y}_i$
1	0	0	0	0	0	$\overline{Y}_0$
1	0	0	0	0	1	$\overline{Y}_1$
1	0	0	0	1	0	$\overline{Y}_2$
1	0	0	0	1	1	$\overline{Y}_3$
1	0	0	1	0	0	$\overline{Y}_4$
1	0	0	1	0	1	$\overline{Y}_5$
1	0	0	1	1	0	$\overline{Y}_6$
1	0	0	1	1	1	$\overline{Y}_7$

一、程序控制的输入/输出（无条件传送）



IO0要输出低电平:

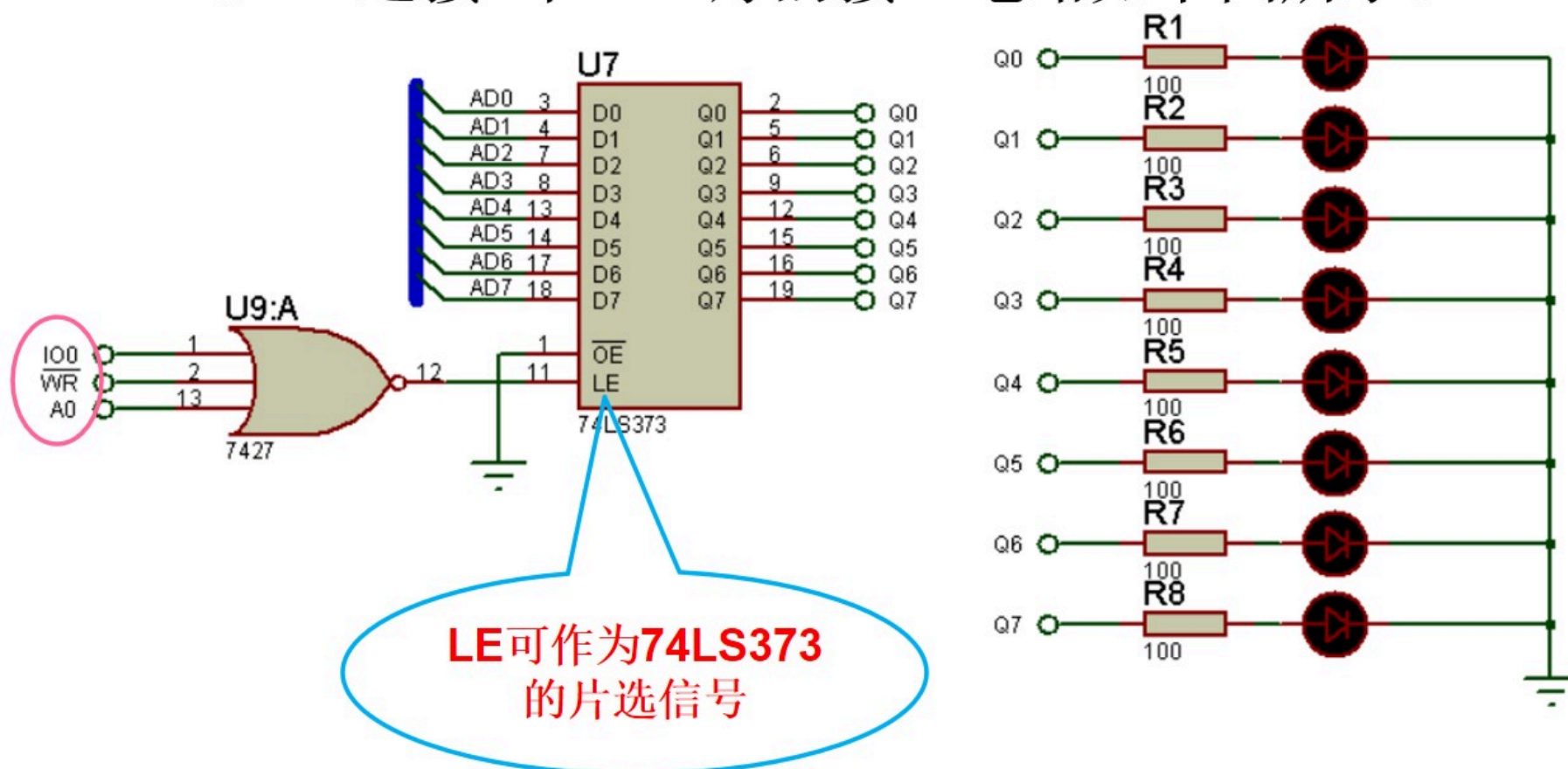
A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
1	1	1	1	1	1	1	1	1	0	0	0				

一、程序控制的输入/输出（无条件传送）



② 74LS373芯片端口地址的分析

74LS373连接8个LED灯的接口电路如下图所示：



一、程序控制的输入/输出（无条件传送）



74LS373的口地址：

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
1	1	1	1	1	1	1	1	1	0	0	0	X	X	X	0

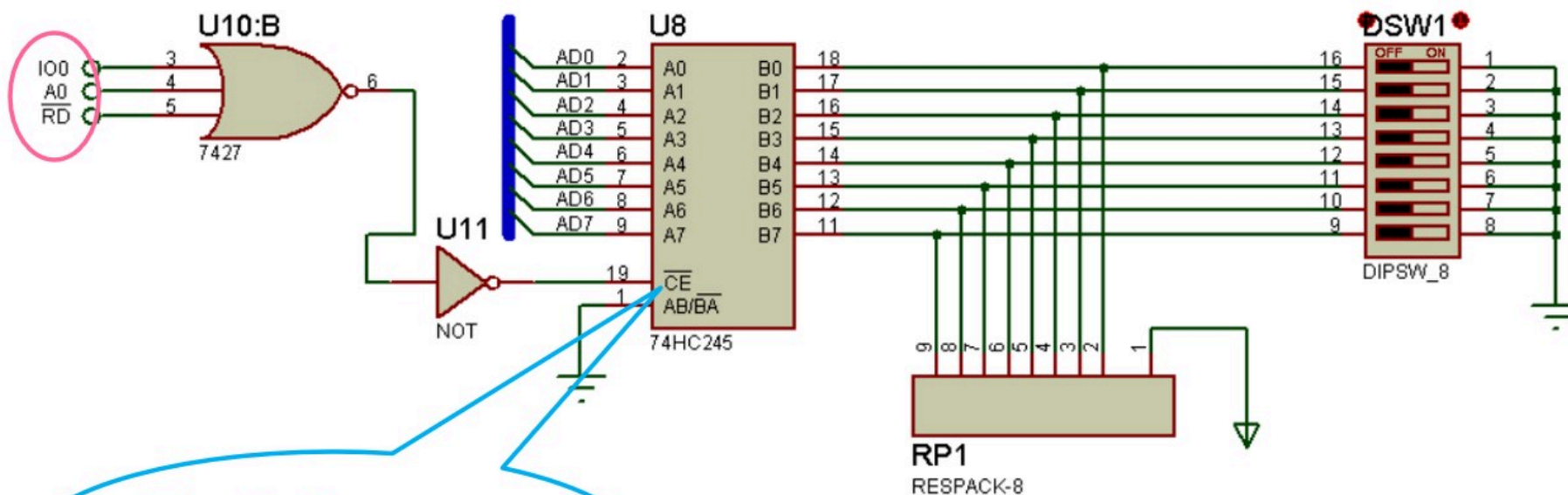
设 $A_3=0, A_2=0, A_1=0$, 则74LS373的口地址为**0FF80H**

一、程序控制的输入/输出（无条件传送）



③ 74HC245芯片端口地址的分析

74HC245连接8个开关的接口电路如下图所示:



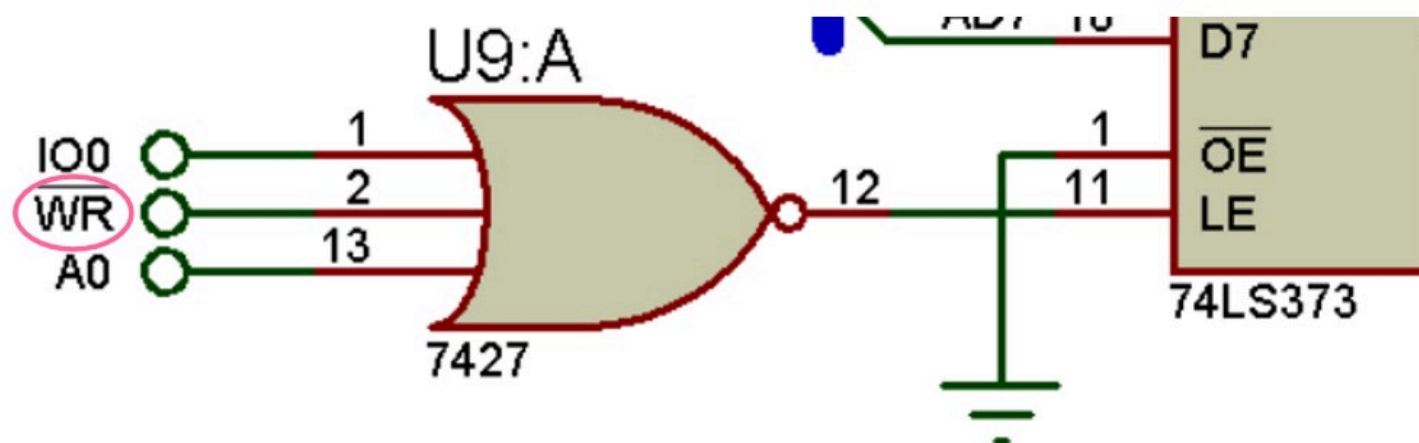
**$\overline{\text{CE}}$ 可作为74HC245
的片选信号**

74HC245的口地址也为0FF80H

一、程序控制的输入/输出（无条件传送）

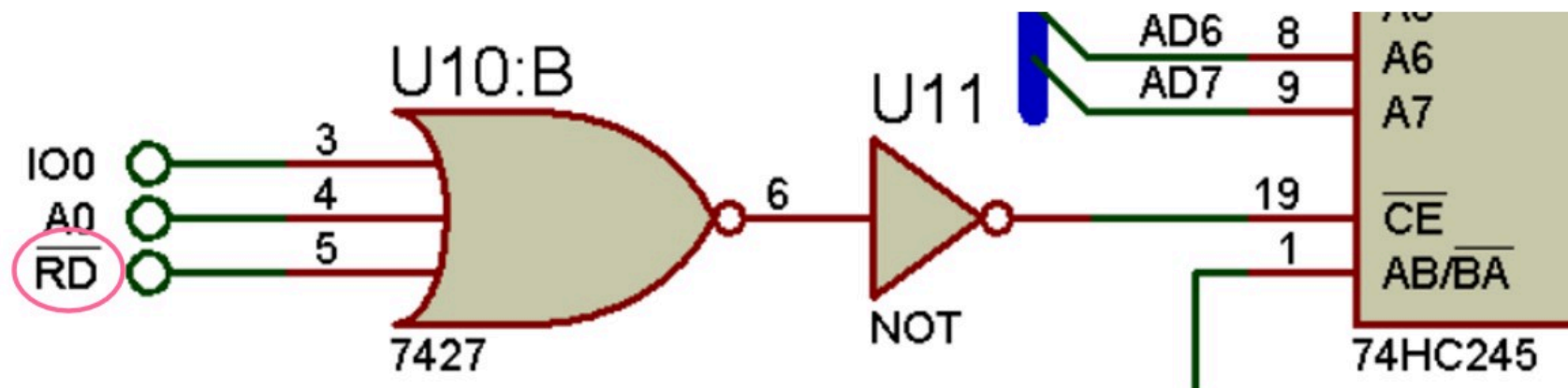


问题：74LS373和74HC245的口地址均为0FF80H，
如何区分CPU访问的是哪个芯片？



CPU写端口0FF80H，访问的是74LS373

一、程序控制的输入/输出（无条件传送）

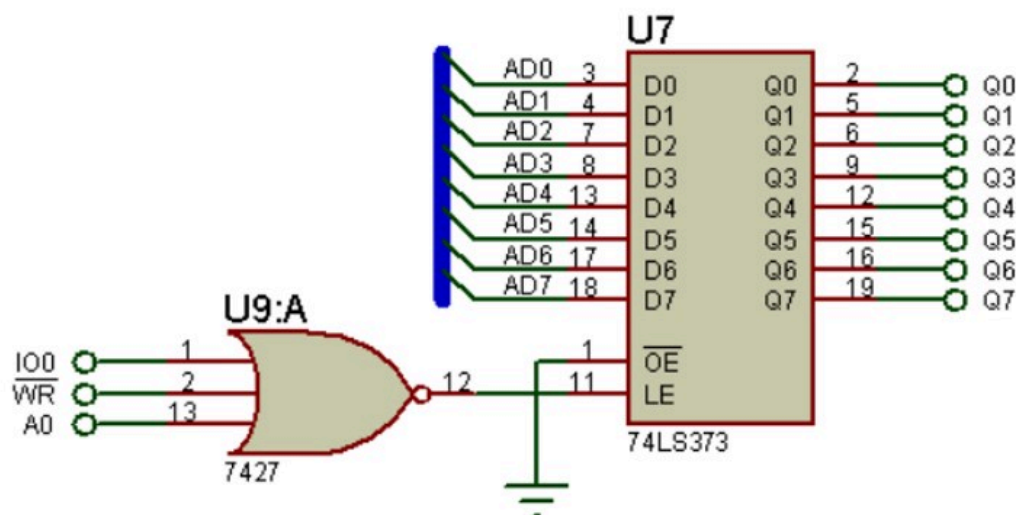


CPU读端口0FF80H，访问的是74HC245

一、程序控制的输入/输出（无条件传送）



④ 输出分析

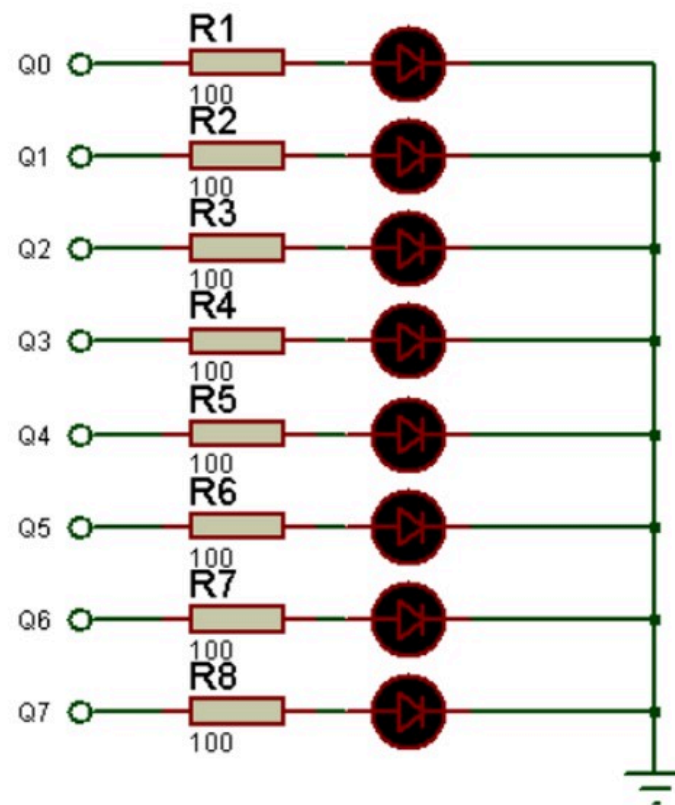


编写指令序列设置8个LED灯全亮:

```
MOV AL,11111111B
```

```
MOV DX,0FF80H
```

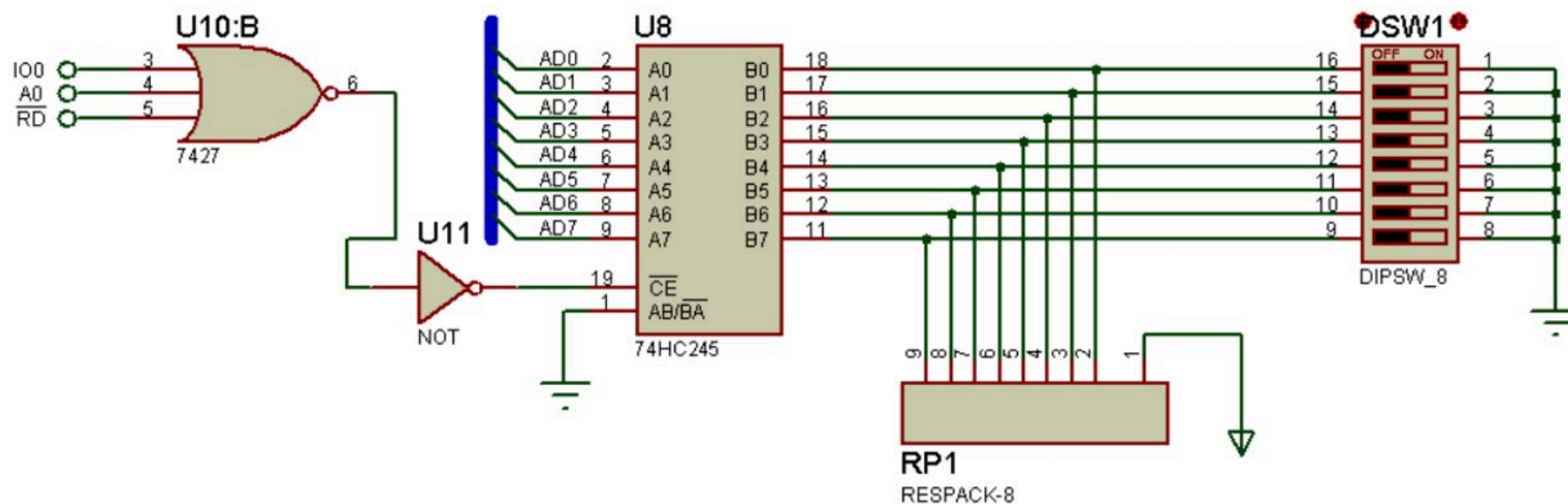
```
OUT DX,AL
```



一、程序控制的输入/输出（无条件传送）



⑤ 输入分析



编写指令序列读取开关的状态：

```
MOV DX,0FF80H
```

```
IN AL,DX
```

一、程序控制的输入/输出（无条件传送）



⑥ 接口程序编写

编写接口程序实现每个开关控制一个**LED**灯。

```
OUT373      EQU    0FF80H
IN245       EQU    0FF80H
CODE SEGMENT
            ASSUME  CS:CODE
START:      MOV     DX,IN245
            IN      AL,DX
            NOT     AL
            MOV     DX,OUT373
            OUT     DX,AL
            JMP     START

CODE ENDS
            END     START
```

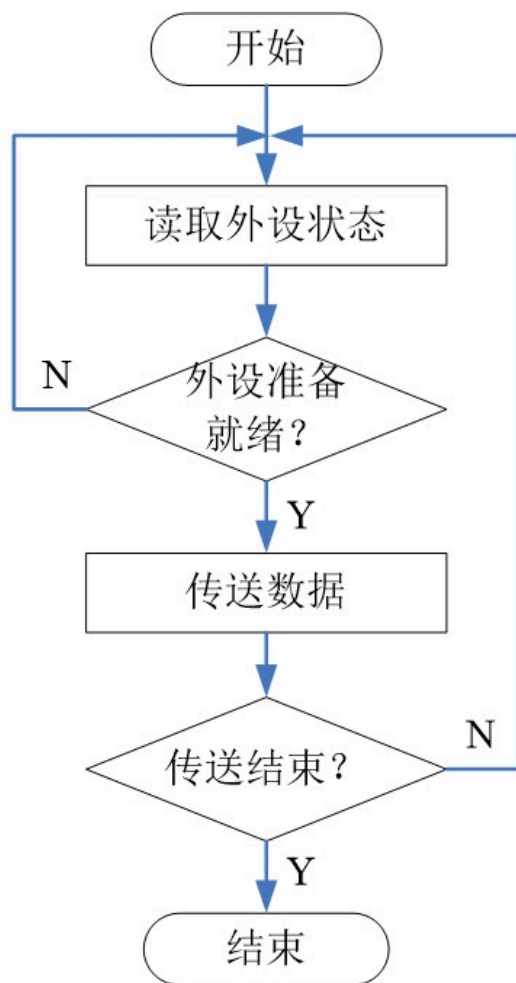

一、程序控制的输入/输出



2、查询方式传送

- **CPU**通过程序指令不断询问外设的工作状态，如果“就绪”就开始进行数据传送，如果未“就绪”则继续不断地询问
- 又称应答式传送，在**CPU**和外设之间建立了问答机制
- 除了使用数据口外，还要用到状态口

一、程序控制的输入/输出（查询方式传送）



工作过程:

- ① 读取I/O接口中的状态口，得到外设的状态字；
- ② 检测相应的状态位，以检查是否“就绪”；
- ③ 若“就绪”则执行I/O操作，若未“就绪”，则重复①~②步。

一、程序控制的输入/输出（查询方式传送）



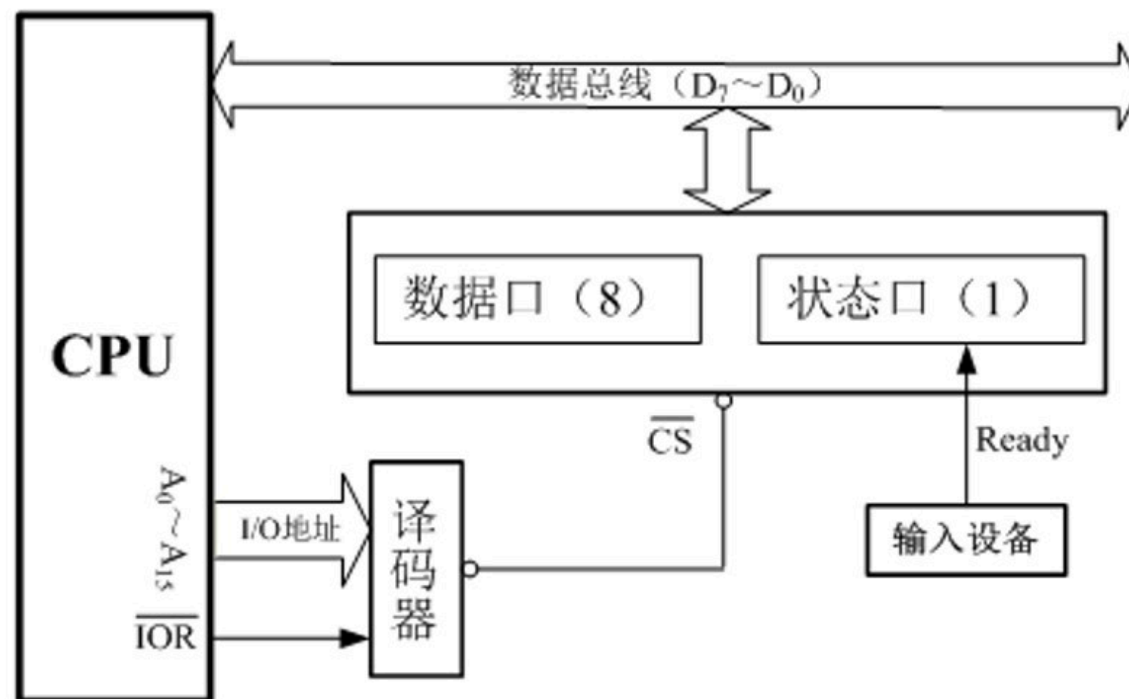
优点：

- ✓ 能保证**CPU**与外围设备之间的协调同步工作
- ✓ 硬件线路简单，程序容易实现

缺点：

- ✓ 浪费**CPU**的时间，效率低

一、程序控制的输入/输出（查询方式传送）



(a) 查询式输入接口模型

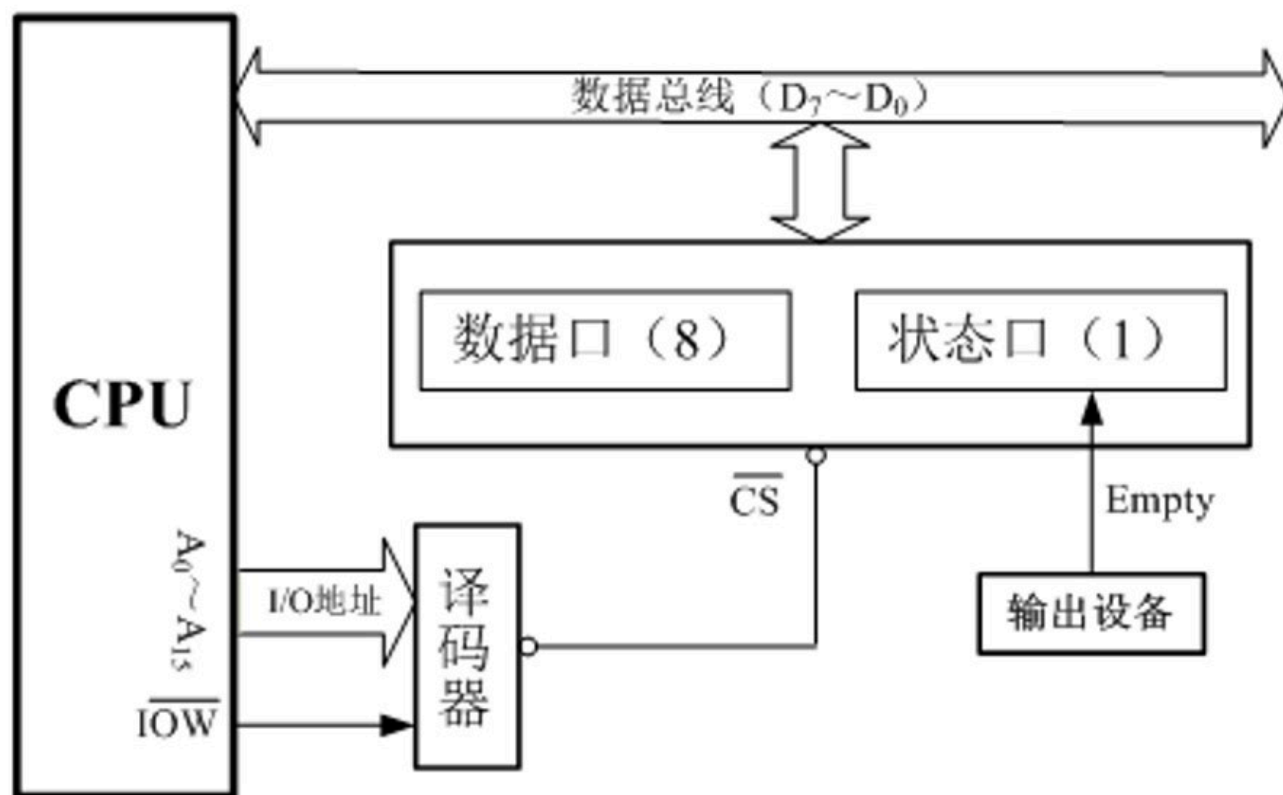
一、程序控制的输入/输出（查询方式传送）



例2：某外设接口的**8**位数据端口地址为**200H**，状态端口地址为**201H**，状态口中第**7**位为**1**表示外设已准备好。如该外设为输入设备，试编制从该设备输入一个字节数据的程序段。

MOV DX, 201H	； 状态口地址
LOP: IN AL, DX	； 读取外设的状态字
TEST AL, 10000000B	； 状态位第7位是否为1？
JZ LOP	； 不是1则继续查询
DEC DX	； 数据口地址
IN AL, DX	； 从数据口输入数据

一、程序控制的输入/输出（查询方式传送）



(b) 查询式输出接口模型

一、程序控制的输入/输出（查询方式传送）



例3：某外设接口的8位数据端口地址为**200H**，状态端口地址为**201H**，状态口中第7位为**1**表示外设已准备好。如该外设为输出设备，则向该设备输出一个字节数据的程序段如何编制？

MOV DX, 201H	； 状态口地址
LOP: IN AL, DX	； 读取外设的状态字
TEST AL, 10000000B	； 状态位第7位是否为1？
JZ LOP	； 不是1则继续查询
DEC DX	； 数据口地址
OUT DX, AL	； 向数据口输出数据

一、程序控制的输入/输出



3、中断方式传送

- 外设“准备好”才发中断，效率高，实时性好
- **CPU**处于被动地位，外设处于主动地位
- 是一种相对可靠的输入/输出方式
- **CPU**与外设“并行”工作，多个外设也可“并行”工作

8.3 基本输入/输出方法（重难点）



一、程序控制的输入/输出（重难点）

二、直接存储器存取方式（DMA）

二、直接存储器存取方式（DMA）



- 程序控制的输入/输出方式都是在**CPU**的干预下通过执行**IN/OUT**指令实现，传送速度不高。
- 直接存储器存取方式（**DMA**）可以在存储器和**I/O**设备间直接传送数据，但需要**DMA**控制器的支持。